

# 4.15

## 1.引入

### 退化问题

在 ResNet 出现之前，深度学习研究者普遍认为，增加网络层数能提升模型性能。然而，实验发现当网络层数超过某个阈值后，性能不升反降。这并非源于过拟合，因为模型在训练集上的误差也在同步增加。

这个被称为 "退化问题" (Degradation Problem) 的瓶颈表明，更深的网络反而更难优化。

☀ 理论上，50层的网络应该效果要比30层的网络好，即便效果没有提升，更深度的网络也应该对浅层的网络做恒等映射，至少效果不会下降，但实际并非如此。

为什么会出现退化问题？

传统神经网络很难做到恒等映射：

- 对于层  $y = \sigma(Wx + b)$ ，要使输出  $y$  精确等于输入  $x$ ：
  - 若激活函数是 ReLU：权重  $W$  必须是对角线上为正、其余为零的稀疏矩阵，且偏置需严格配合输入分布。
  - 若激活函数是 Sigmoid/Tanh：还需让输入落在激活函数的线性区（导数接近1的狭窄区间）。

随着网络深度增加，丢失很多信息：

- 每一层的非线性变换（如 ReLU 的置零操作）会不可逆地丢弃部分输入信息。
- 经过多层堆叠，输入信号中能传播到深层的信息自由度不断降低。到最后一层时，权重对输入的微小变化变得不敏感，导致反向传播的梯度虽然数值存在，但指向的更新方向对改进浅层表示作用甚微。
- 这意味着：深层网络的实际可用容量并没有随层数线性增长，反而因信息瓶颈而衰减。当衰减超过新增层带来的理论增益时，退化便发生了。

### 梯度消失/爆炸

核心原因是链式连乘 + 激活函数导数过小：

- 假设网络深度为  $L$ ，第  $l$  层的梯度是损失对该层权重的偏导：

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} \propto \prod_{k=l+1}^L \frac{\partial a^{(k)}}{\partial a^{(k-1)}}$$

- 若使用 Sigmoid 或 Tanh 激活函数，其导数最大值分别仅 0.25 和 1，且输入稍大或稍小时导数趋近于 0。多个小于 1 的数连乘，指数级衰减。
- 权重初始化不当（如初始值过小）会进一步放大该效应。

图1.1 梯度的链式连乘

推导：

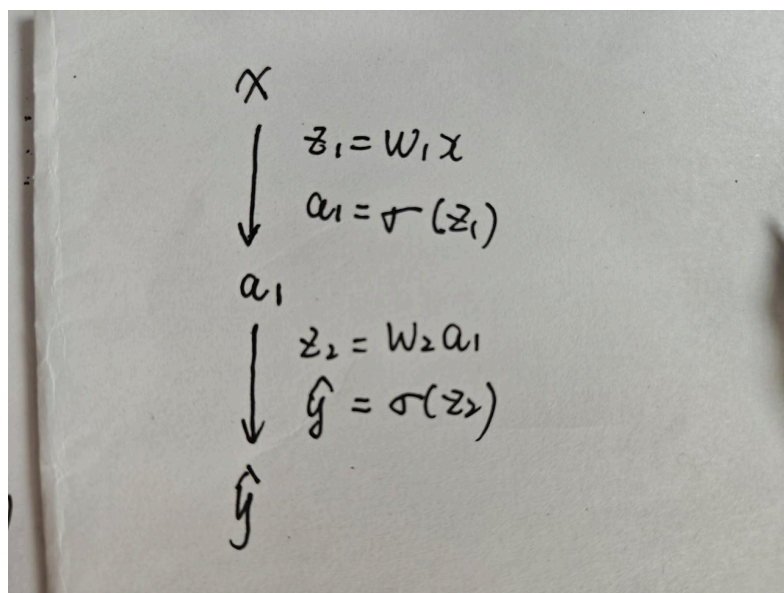


图1.2 一个简单的网络

$\sigma$  是损失函数。

$w_1$ ， $w_2$  是权重。

$w_2$  影响  $z_2$ ， $z_2$  影响  $\hat{y}$ ， $\hat{y}$  影响  $L$ 。

根据链式法则：

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2}$$

$w_1$  影响  $z_1$ ， $z_1$  影响  $a_1$ ， $a_1$  影响  $z_2$ ， $z_2$  影响  $\hat{y}$ ， $\hat{y}$  影响  $L$ 。

链式法则展开：

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial w_2} \cdot \left( \frac{1}{a_1} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot x \right)$$

图1.3 链式推导

可以看出，反向传播过程中，浅层网络的梯度由深层网络的梯度逐渐递推得出，是一个连乘形式，那么当梯度大于1时，计算出的浅层网络梯度会在这样的连乘变得很大，当梯度小于1时，又会变得很小，趋于0。

这让我们在进行权重更新时，浅层的参数波动要不就是很大，要不就是几乎没有改动，影响模型精确度。

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \frac{\partial \text{Loss}}{\partial W}$$

图1.4 权重更新

## 2.ResNet的贡献

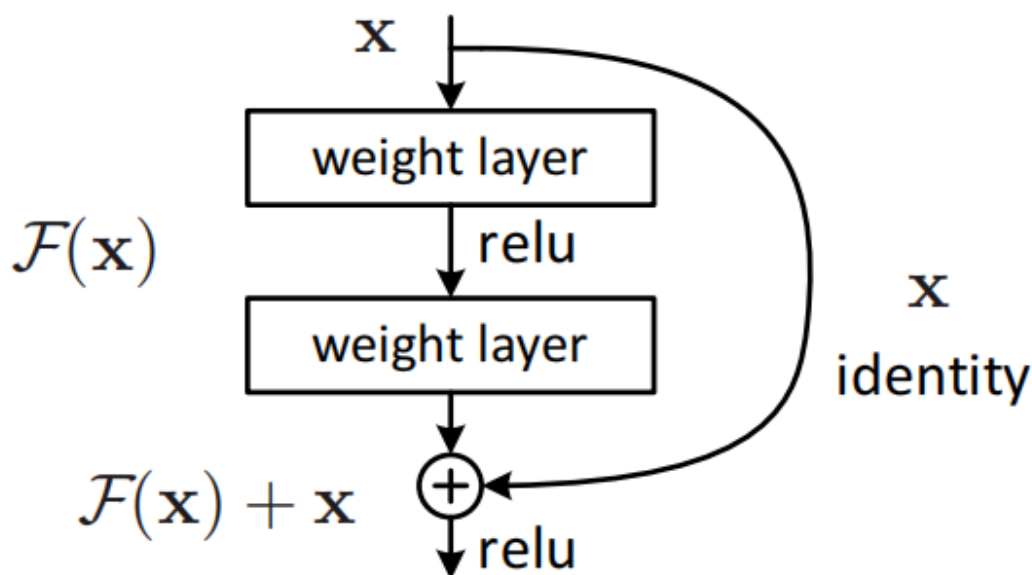


图2.1 Residual learning: a building block

原先：输入  $x$ ，经过映射  $H(x)$ ，输出。

改动：输入  $x$ ，经过映射  $F(x)$ ，再加上  $x$ ，输出。

代码部分上来看，仅多了：

代码块

```
1 out += identity
```

## 关键区别：加性梯度 vs 乘性衰减

在残差块的梯度公式中：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \left( 1 + \frac{\partial F}{\partial x} \right) = \underbrace{\frac{\partial L}{\partial y}}_{\text{恒等映射分支}} + \underbrace{\frac{\partial L}{\partial y} \cdot \frac{\partial F}{\partial x}}_{\text{残差分支}}$$

- “+1”的作用：它保证无论残差分支的梯度  $\frac{\partial F}{\partial x}$  多小（甚至为 0），恒等映射分支始终能将深层梯度原样传递，即  $\frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial y}$ 。梯度不会因连乘而指数衰减到零。

图2.2 ResNet 对原本的改进细节

这样可以近似做到恒等映射。

## 3.运行代码 debug

ResNet-34

```
# Stem 部分: 7x7 卷积, 步长 2, padding 3, 输出通道 64
self.conv1 = nn.Conv2d(3, self.in_channel, kernel_size=7, stride=2,
|   |   |   |   |   |   | padding=3, bias=False)
self.bn1 = nn.BatchNorm2d(self.in_channel)
self.relu = nn.ReLU(inplace=True)
# 3x3 最大池化, 步长 2, 进一步下采样
self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

# 构建 4 个残差层, 每个 layer 包含多个残差块
# layer1: 输出通道 64, 步长 1 (不下采样), blocks_num[0] 个块
self.layer1 = self._make_layer(block, 64, blocks_num[0])
# layer2: 输出通道 128, 第一个块步长 2 (下采样), 其余步长 1
self.layer2 = self._make_layer(block, 128, blocks_num[1], stride=2)
# layer3: 输出通道 256, 第一个块步长 2
self.layer3 = self._make_layer(block, 256, blocks_num[2], stride=2)
# layer4: 输出通道 512, 第一个块步长 2
self.layer4 = self._make_layer(block, 512, blocks_num[3], stride=2)
```

图3.1 部分 code

```

# 前向传播
def forward(self, x):
    # 保存输入作为捷径连接的初始值
    identity = x
    # 如果需要下采样，则对捷径进行变换，使尺寸和通道数匹配主分支的输出
    if self.downsample is not None:
        identity = self.downsample(x)

    # 主分支: conv1 -> bn1 -> relu
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)

    # 主分支: conv2 -> bn2
    out = self.conv2(out)
    out = self.bn2(out)

    # 将主分支输出与捷径连接相加（残差连接）
    out += identity
    # 最后经过 ReLU 激活
    out = self.relu(out)

    return out

```

图3.2 部分 code

运行结果：

```

train epoch[1/3] loss:0.371: 100%| 207/207 [00:14<00:00, 14.00it/s]
valid epoch[1/3]: 100%| 23/23 [00:00<00:00, 31.08it/s]
[epoch 1] train_loss: 0.501 val_accuracy: 0.929
train epoch[2/3] loss:0.139: 100%| 207/207 [00:10<00:00, 18.91it/s]
valid epoch[2/3]: 100%| 23/23 [00:00<00:00, 35.94it/s]
[epoch 2] train_loss: 0.318 val_accuracy: 0.942
train epoch[3/3] loss:0.295: 100%| 207/207 [00:11<00:00, 18.75it/s]
valid epoch[3/3]: 100%| 23/23 [00:00<00:00, 26.37it/s]
[epoch 3] train_loss: 0.293 val_accuracy: 0.945
Finished Training

```

图3.3 运行结果

学习的新东西：

- 批归一化层：概括之，把这一批数据的输出变为到均值0、方差1的标准状态。用两个可学习的参数 ( $\gamma$  和  $\beta$ ) 做缩放和平移

## 4.一点验证

删去代码：

#### 代码块

```
1 out += identity
```

```
train epoch[1/3] loss:1.431: 100%| 207/207 [00:12<00:00, 16.93it/s]
valid epoch[1/3]: 100%| 23/23 [00:00<00:00, 28.46it/s]
[epoch 1] train_loss: 1.596 val_accuracy: 0.266
train epoch[2/3] loss:1.514: 100%| 207/207 [00:10<00:00, 19.32it/s]
valid epoch[2/3]: 100%| 23/23 [00:00<00:00, 36.87it/s]
[epoch 2] train_loss: 1.559 val_accuracy: 0.330
train epoch[3/3] loss:1.435: 100%| 207/207 [00:12<00:00, 16.10it/s]
valid epoch[3/3]: 100%| 23/23 [00:00<00:00, 32.36it/s]
[epoch 3] train_loss: 1.551 val_accuracy: 0.313
Finished Training
```

图3.4 运行结果

对之前学习过的 CNN 代码改动：

```
def forward(self, x):
    """
    定义前向传播过程
    """
    # 输入张量 x 的形状应为 (batch_size, 1, 28, 28)
    # 其中 batch_size 是批次大小, 1 表示单通道灰度图, 28x28 是图像的空间维度
    identity = x # 保存输入张量以用于残差连接
    x = self.pool(F.relu(self.conv1(x))) # 第一个卷积层 -> ReLU 激活函数 -> 最大池化层
    x += identity # 添加残差连接, 将输入直接加到卷积层的输出上
    identity = x # 更新残差连接的输入
    x = self.pool(F.relu(self.conv2(x))) # 第二个卷积层 -> ReLU 激活函数 -> 最大池化层
    x += identity # 添加残差连接, 将输入直接加到卷积层的输出上
    x = x.view(-1, 64 * 7 * 7) # 将张量展平成一维向量, -1 表示自动计算 batch size
    x = F.relu(self.fc1(x)) # 第一个全连接层 -> ReLU 激活函数
    x = self.fc2(x) # 第二个全连接层, 输出最终的分类结果
    return x # 返回输出
```

图3.5 code

结果：

印证原论文，在浅层网络使用该方法也没有问题。